DESIGN DOCUMENT

AI-Powered Competitive Intelligence

& Campaign Strategy System

1. Purpose & Problem Statement

Competitive research is manual, inconsistent, and rarely translates directly into campaign action. Teams spend hours monitoring sources, synthesizing signals, and drafting strategy — work that is both repetitive and high-stakes.

This system automates that pipeline end-to-end: from scraping competitor updates to delivering structured intelligence and a phase-wise GTM strategy through a real-time dashboard. The output is not just data — it is a decision-ready artifact for product and marketing teams.

2. System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                       INGESTION LAYER                        │
│    Celery Beat (Scheduler) → Redis Queue → Scrape Workers    │
└──────────────────────────────────────────────────────────────┘
                              │  Raw HTML / Article Text
                              ▼
                     ┌──────────────────┐
                     │   PostgreSQL DB  │
                     │  Raw + Processed │
                     └──────────────────┘
                              │  Structured Records
                              ▼
                 ┌──────────────────────────┐
                 │     Django REST Framework │
                 │  Analysis + Retrieval Endpoints│
                 └──────────────────────────┘
                              │  Prompt + Content Block
                              ▼
                 ┌──────────────────────────┐
                 │      Gemini LLM (API)     │
                 │   Structured JSON Response │
                 └──────────────────────────┘
                              │  Persisted Insights
                              ▼
                 ┌──────────────────────────┐
                 │      Next.js Dashboard    │
                 │  Signals · Pain Points · GTM │
                 └──────────────────────────┘
```

Component Responsibilities

Component	Technology	Role
Task Scheduler	Celery Beat + Redis	Queue and throttle scrape jobs on a defined cadence
Scraper Workers	Celery Tasks (Python)	Fetch and extract article content from target URLs

Component	Technology	Role
Database	PostgreSQL	Persist raw content, deduplication metadata, and LLM output
Backend API	Django + DRF	Orchestrate analysis requests; serve insights to frontend
AI Engine	Gemini API	Extract competitive signals; generate GTM strategy
Frontend	Next.js	Render structured insights in card-based dashboard UI

3. Data Flow (Step-by-Step)

- **1. Schedule:** Celery Beat enqueues a scrape job at a configured interval; Redis holds the task queue.
- **2. Scrape:** Worker fetches target URLs, strips HTML boilerplate, extracts article text, title, and publish date.
- **3. Deduplicate & Store:** Raw content is written to PostgreSQL; duplicates are dropped on URL + normalized date.
- **4. Trigger Analysis:** A DRF endpoint receives an analysis request, queries unprocessed records, and builds a prompt payload.
- **5. LLM Call:** The backend sends a structured prompt + content block to the Gemini API.
- **6. Parse & Validate:** The JSON response is schema-validated; missing or malformed fields default to empty arrays — the pipeline does not fail.
- **7. Persist Insights:** Structured output is written back to PostgreSQL, linked to source article IDs for traceability.
- **8. Serve:** DRF endpoints expose insights by category; Next.js fetches and renders them in the dashboard.

4. AI Prompt Design

The system uses a single structured prompt combining signal extraction and strategy synthesis. Output is schema-constrained to JSON — this is a deliberate choice to make LLM output directly renderable without a transformation layer.

```
SYSTEM:
You are a competitive intelligence analyst. Analyze the competitor
content below. Return ONLY valid JSON matching the schema.
Do not include explanation or markdown.

SCHEMA:
{
  "competitive signals": {
    "features in development": [...],
    "messaging_trends": [...],
    "customer_pain_points": [...],
    "weaknesses": [...]
  },
  "campaign strategy": {
    "messaging_strategy": "...",
    "recommended_channels": [...],
    "gtm_phases": {
      "pre launch": "...",
      "launch": "...",
      "growth": "..."
    }
  }
}
```

```
INSTRUCTION:
Only infer from the provided content. Return empty arrays or strings
where data is insufficient. Do not fabricate signals.

CONTENT: {scraped_articles_block}
```

Prompt Reasoning

- Combining extraction and synthesis in one call minimizes latency and token overhead.
- Schema-first prompting removes the need for post-processing logic on the backend.
- The 'do not fabricate' instruction is a direct hallucination mitigation measure, not a formality.

5. Handling Noisy & Unstructured Data

Web-scraped content is inconsistent by nature. The following mitigations are applied at each stage:

Problem	Mitigation
Navigation / boilerplate text	Strip HTML; extract <article> or main content blocks only
Duplicate articles	Deduplicate on URL + normalized publish date before storage
Paywalled or empty content	Skip on empty body extraction; log for manual review
LLM hallucination	Prompt instructs inference from content only; empty fields over invented ones
Malformed JSON response	Backend validates schema; missing keys default gracefully
Conflicting signals across articles	LLM consolidates across full context window; conflicts surface as distinct list entries

Known gap: Articles are currently treated with equal weight regardless of source credibility. Source quality scoring is a near-term improvement.

6. Design Decisions & Trade-offs

Decision	Chosen Approach	Rationale
Data sourcing	Web scraping	No API access required; sufficient for public signals at prototype scale; lower cost than paid intelligence feeds
Reasoning engine	LLM (Gemini)	Rule-based extraction cannot generalize across varied, unstructured competitor content; LLMs handle semantic inference naturally
Processing model	Batch (Celery)	Real-time streaming is unnecessary for intelligence consumed once or twice daily; batch reduces API cost and complexity
Output format	Structured JSON	Forces consistent, renderable output; eliminates a post-processing layer and makes frontend integration trivial
Backend framework	Django + DRF	Mature ORM, built-in admin, straightforward API scaffolding — appropriate for a data-heavy backend

7. Assumptions

- Competitor updates published on tech news sources reflect actual product and marketing direction.
- Article titles and body text contain sufficient signal for trend extraction — full roadmaps are not required.
- The LLM can generalize meaningful patterns from a limited batch of 10–30 articles per run without large-scale training data.
- Marketing and product teams check insights asynchronously — near-real-time delivery is not a hard requirement at this stage.

8. Why This System Matters

- **Product Teams:** Surfaces feature signals from competitor coverage before they become widely known, enabling earlier roadmap responses.
- **Marketing Teams:** Translates competitor data directly into a messaging strategy and GTM plan — replacing a multi-hour manual process with a dashboard refresh.
- **Leadership:** Reduces reliance on ad hoc competitive research; creates a consistent, repeatable intelligence cadence.
- **Operational Impact:** A task that previously required 4–6 hours of analyst time per week is reduced to minutes, with higher consistency and auditability.

9. Evaluation & Validation

- **Structural Consistency:** LLM outputs were validated across multiple runs to confirm schema compliance and field completeness.
- **Signal Accuracy:** Extracted features and messaging trends were cross-referenced against known competitor activity to verify alignment.
- **Strategy Coherence:** Generated GTM phases were reviewed against standard product launch frameworks (JTBD, RACE) to confirm logical sequencing.
- **Failure Handling:** Edge cases — empty content, paywalled articles, malformed JSON — were tested explicitly; pipeline degrades gracefully in all cases.

10. Scalability Considerations

The current architecture is designed for prototype scale. Scaling paths are well-defined:

- **More Competitors:** Scraper targets are configuration-driven; adding a new source requires a URL entry, not code changes.
- **Higher Data Volume:** PostgreSQL handles significant load; read replicas and indexing on article date + source are the first scaling levers.
- **Streaming Pipeline:** Celery batch jobs can be replaced with Kafka or Celery chains if near-real-time processing becomes a requirement.
- **AI Cost Control:** LLM responses for identical content windows can be cached in Redis or the DB layer to avoid redundant API calls.
- **Horizontal Scaling:** Celery workers are stateless and horizontally scalable; worker count can increase without architectural changes.

11. Limitations

- **Narrow Data Sources:** Only public web sources are scraped; no social signals, review platforms, job postings, or sales intelligence are included.
- **LLM Output Variability:** Gemini responses are non-deterministic; two runs on identical content may produce slightly different phrasings without output caching.
- **No Temporal Tracking:** Insights are point-in-time snapshots. Trend evolution over 90 days is not currently tracked or visualized.
- **No Personalization:** All users see the same intelligence output; no filtering by vertical, segment, or company-specific context.
- **Scraper Fragility:** DOM changes on target sites break extractors silently. There is no automated scraper health monitoring in the current build.

12. If More Time Was Available

These are the highest-priority improvements, ordered by product impact:

- **1. Expanded Data Sources:** Integrate LinkedIn posts, Twitter/X, G2/Capterra reviews, and competitor job postings. Job postings are a strong proxy for where a competitor is investing product resources.
- **2. Trend Evolution Tracking:** Store insights with timestamps and build a diff layer to surface how competitor messaging and positioning shift over time — turning snapshots into longitudinal intelligence.
- **3. Use-Case Personalization:** Allow teams to configure their company profile, target segment, and competitive context so the LLM generates tailored, not generic, strategy output.
- **4. Prompt Refinement & Chaining:** Separate signal extraction and strategy synthesis into two chained calls. Use a smaller model for extraction and a more capable model for strategy — reducing cost without sacrificing quality.
- **5. Production Readiness:** Add scraper health monitoring, structured logging, error dashboards, and LLM response caching. Deploy with a managed worker pool and environment-based configuration for staging and production parity.